

Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1	Introduction to Embedded Systems and Toolchain	5
1.1	The Chain of Evidence and the Bare-Metal Toolchain	5
1.2	What is an Embedded Operating System?	6
1.2.1	Key Properties of Embedded Systems	6
1.3	Operating System Architectures	7
1.3.1	Monolithic OS Structure	7
1.3.2	Microkernel OS Structure	7
1.3.3	Flat OS Structure (Bare-Metal)	8
1.3.4	What we will do in the course?	8
1.4	The Target Machine: Cortex-M3 and QEMU	9
1.4.1	Microcontroller (MCU) vs General-Purpose CPU	9
1.4.2	The QEMU target (<code>mps2-an385</code>)	9
1.4.3	Peripheral Registers and the <code>volatile</code> Keyword	9
1.5	Technical Foundations & Toolchain	10
1.5.1	From Source Code to a Running Program	10
1.5.2	Why Cross-Compilation?	10
	Checking the Toolchain: Host vs Target	11
1.5.3	Host, Target, and Emulator	11
1.5.4	Reset Code and the Transition to C	11
1.5.5	Preprocessing: Assemble the Translation Unit	12
	Try It: Preprocess Only	12
1.5.6	Compilation: C Becomes Target Instructions	13
	Compilation and Target Architecture	13
	Try It: Stop After Compilation	13
1.5.7	Assembly: Instructions Become Relocatable Bytes	13
	Try It: Assemble to an Object File	14
	Inspecting Object Files	14
1.5.8	Linking: Creating the Executable	14
	The Linker Script	15
	Try It: Link the Objects	15
	The Linker Script and the Map File	15
1.5.9	The ELF Image and its Map	16
	Thumb State and The Entry Point	16
	Sections Layout in Week 1	16
	File Size vs RAM Usage	16
1.5.10	Debug Information for Host Tools	17
1.5.11	What Make Actually Does	17
	Targets Model Desired Outcomes	17

Prerequisites Capture Rebuild Logic	17
1.5.12 Variables and Compiler Flags	18
Build Directories Keep Artifacts Separate	18
Variables and String Manipulation in Make	19
1.5.13 Defining Targets and Rules	19
Diagnosing the "Missing Separator" Error	20
Variable Assignment Types and Debugging	20
Automatic Variables	21
Pattern Rules and Substitution References	21
Phony Targets	21
Reading the Week 1 Makefile as a Dependency Graph	21
A Disciplined First Build Loop	22
1.5.14 QEMU Runs a Board Model	22
1.5.15 QEMU, Semihosting, and the Observed Result	23
Why Avoid <code>printf</code> ?	23
Running the Demonstration	23
Interpreting the Output	23
The Final Spin Loop and QEMU Execution	24
1.6 A Practical Inspection Routine	24
1.6.1 Systematic Troubleshooting	24
1.6.2 Foundational Significance	25
2 Boot and CPU Configuration	27
2.1 Introduction to Low-Level CPU Control	27
2.1.1 Where You Start and Where You End	27
2.2 Memory Technologies in Embedded Systems	28
2.2.1 SRAM (Static RAM): The Working Memory	28
2.2.2 NOR Flash: Execute-in-Place Memory	28
2.2.3 NAND Flash and Block Devices	29
2.2.4 Applying Constraints to C Code	29
2.2.5 The Cortex-M3 Memory Map	30
2.2.6 The Linker Script: Mapping the Chip	30
Defining the Memory Regions	30
Defining the Memory Regions (Continued)	31
Symbol Assignments for the Stack Pointer	31
Organizing Output Sections	31
2.2.7 Run Address and Load Address: The <code>.data</code> Section	32
The Linker Does Not Copy Data	33
2.2.8 BSS, Heap, and Stack Layout	33
Safety Asserts in the Linker Script	33
2.2.9 The Role of Linker-Defined Symbols	33
Memory Alignment	34
2.2.10 Reading the Compilation Results	34
Analyzing the Base Image	34
2.2.11 Which Section Does a Declaration Go To?	35
Pointers, Local, and Static Variables	35
Try It: Analyzing the Output and Little-Endianness	35
Pointers in RAM and Little-Endianness	36

2.2.12	Advanced Memory Layout: Custom Sections and Orphans . . .	36
	Forcing Variables into Specific Sections	37
2.2.13	Overflowing a Region	37
2.2.14	Reset: The Hardware Contract	37
2.2.15	The Vector Table: Controlling the Boot Process	38
	Implementing the Vector Table in Assembly	38
	The Thumb Bit Constraint	38
	<i>KEEP</i> and Garbage Collection	39
2.2.16	Back to the Base: Why Only One Line Prints	39
2.3	Taking Control of the CPU	39
2.3.1	Startup Code: Assembly vs. C	39
2.3.2	What is a Debugger?	40
2.3.3	A Remote Debugger: Two Programs, Two Terminals	40
	GDB: The GNU Debugger	40
	Setting Up the Debugging Session	41
	Reconstructing the Debug Target	41
	The GDB Commands You Need	42
	Attaching at Reset	42
2.3.4	Cortex-M3 Programmer's Model: The Registers	42
	The Status Register (xPSR) and CPU Modes	43
2.3.5	ARM Assembly Basics: A Load/Store Architecture	43
	Constants, Labels, and Jumps	44
2.3.6	Control Flow: Decisions, Bits, and Interrupts	44
2.3.7	Assembler Directives	45
2.4	Writing the Startup Script (<code>Reset_Handler</code>)	45
2.4.1	Steps 1 and 2: Mask Interrupts and Set the Stack	46
2.4.2	Step 3: Copy <code>.data</code> from FLASH to RAM	46
	Try It: Bring Data to Life	47
	Watching the Copy Loop in GDB	47
	Try It: Breaking the Stack on Purpose	47
2.4.3	Step 4: Zero the <code>.bss</code> Section	48
2.4.4	Exceptions and Faults: One Family	48
	Completing the Table with Weak Symbols	49
	How it works:	49
	The Effect on Memory Layout	50
2.4.5	Step 5: Configure the CPU (System Control Block)	50
	Read-Modify-Write and the <code>volatile</code> Keyword	50
	VTOR: Vector Table Offset Register	50
	SHCSR: System Handler Control and State Register	51

Chapter 1

Introduction to Embedded Systems and Toolchain

1.1 The Chain of Evidence and the Bare-Metal Toolchain

Building a bare-metal firmware relies on a strict **chain of evidence**. Every tool in the build pipeline has a specific, essential job to connect your C source code to the physical hardware.

Here is exactly what each component does in practice:

- **Cross-Compiler (e.g., arm-none-eabi-gcc):** Your PC cannot run micro-controller code. The cross-compiler's job is to translate your human-readable C code into the exact binary language that the target CPU understands (for the Cortex-M3, this is the **Thumb-2** instruction set).
- **Linker and Linker Script (.ld):** The compiler creates isolated blocks of code, but the CPU needs them placed at specific physical addresses. The linker glues these blocks together, using the **linker script** as a map. It dictates that executable code and constants must be saved in permanent **Flash** memory, while writable variables must be placed in volatile **SRAM**.
- **Hardware Vector Table:** A fixed directory located at the very beginning of the memory (address 0x00000000). When powered on, the CPU is "blind" and reads the first two entries here to know how to start:
 1. The first word tells the CPU where to place its temporary working memory (the **Main Stack Pointer**).
 2. The second word tells the CPU the exact address of the first instruction to execute (the **Reset Handler**).
- **Startup Code (Reset Handler):** A crucial piece of code that runs *before* your `main()` function. Its job is to "prepare the house" (the RAM) for your application. It performs two critical tasks:
 1. **Data Initialization:** It copies variables that have a starting value (e.g., `int x = 5;`, the `.data` section) from the permanent Flash into the working SRAM.

2. **BSS Clearing:** It fills variables that have no starting value (the `.bss` section) with zeros directly in SRAM.

Only after the RAM is perfectly set up, the startup code safely calls `main()`.

Understanding what each component actually does transforms debugging. If a program crashes before even reaching `main()`, a developer immediately knows to check the linker script or the startup code, rather than the application logic.

1.2 What is an Embedded Operating System?

An operating system (OS) manages hardware resources for applications. General-Purpose OS use a Memory Management Unit (MMU) to isolate applications in virtual memory.

An **Embedded Operating System (EOS)** is completely different: it is a highly specialized, tightly constrained OS built to execute a single, fixed workload on a specific hardware device.

1.2.1 Key Properties of Embedded Systems

- **Resource Constraints:** Built for kilobytes of RAM/Flash and milliwatts of power. Zero overhead allowed.
- **Real-Time Determinism (RTOS):** Deadlines matter as much as logic. A correct calculation delivered late (e.g., in an airbag sensor) is a total system failure. Predictability beats raw speed.
- **Flat Memory Architecture:** No virtual memory. The kernel, drivers, and tasks share one physical address space and are compiled into a single firmware image.
- **Static Configuration:** Since the workload is fixed at compile time, dynamic memory allocation (`malloc`) is rarely used. Tasks, memory pools, and hardware resources are statically allocated.

EMBEDDED VS. GENERAL-PURPOSE		
	Embedded	General-purpose
Workload	one fixed task	anything the user installs
Resources	KB of RAM/flash	GB of RAM, TB of disk
Timing	often hard deadlines	best-effort
User model	no interactive user	multi-user, multi-tasking
Cost/power	pennies, milliwatts	dollars, tens of watts

Figure 1.1: Embedded vs General Purpose OS characteristics.

1.3 Operating System Architectures

1.3.1 Monolithic OS Structure

The entire operating system runs as a single, large program in a privileged kernel space.

- **Unified Kernel:** Core services (scheduling, memory, file systems, drivers) live together.
- **Maximum Throughput:** Components share the same memory and communicate directly without message-passing overhead.
- **The Risk:** If one driver crashes, the entire system halts (e.g., Linux, Windows).

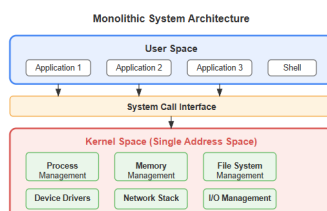


Figure 1.2: Monolithic Operating System Structure.

1.3.2 Microkernel OS Structure

A minimalist design that keeps only essential features (scheduling, basic memory management, IPC) in privileged mode. Everything else runs as isolated user-space apps.

- **Fault Containment (Advantage):** High security and reliability. If a device driver crashes, it restarts independently without bringing down the core OS.
- **Performance Penalty (Disadvantage):** Slower than monolithic systems. Isolated components must constantly talk via Inter-Process Communication (IPC), causing heavy context-switching overhead.

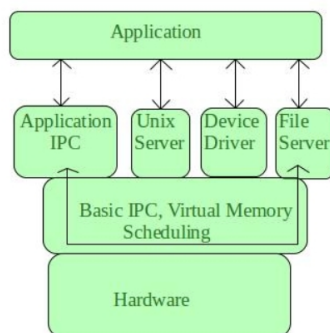


Figure 1.3: Microkernel Operating System Structure.

1.3.3 Flat OS Structure (Bare-Metal)

The simplest architecture, where there are no layers or isolated spaces.

- **Single Linked Image:** The startup code, OS kernel, device drivers, and apps are merged into one executable stored in Flash. No dynamic app loader.
- **Raw Physical Memory:** Everything maps directly to raw physical addresses. No MMU translation.
- **No Hardware Isolation:** No protection boundaries between kernel and user space.

Advantages:

- **High Speed:** Zero latency from OS abstractions or IPC.
- **Simplicity:** Easy to build and deploy for small, dedicated hardware.

Disadvantages & Risks:

- **Zero Isolation:** A single bug (e.g., a bad pointer) in any task can overwrite the entire system memory and cause a hard crash.
- **Blocking Tasks:** A stuck function or infinite loop halts the entire CPU.
- **Poor Scalability:** Unmanageable as code complexity grows.

1.3.4 What we will do in the course?

The starting point (*scratch-os*): The course begins by building *scratch-os*, which utilizes a *flat firmware architecture*. This exposes the entire internal pipeline, from the initial boot process to task scheduling, making the hardware assumptions completely visible and non-negotiable.

The transition (FreeRTOS): Later, we transition to FreeRTOS, a production-ready RTOS closer to a microkernel philosophy. It features a small dedicated kernel and implements real scheduling policies (though, by default, it does not enforce memory protection). The educational goal is clear: *using a high-level RTOS API is easy only when you already understand the invisible state transitions happening underneath it.*

The exact **sequence of core mechanisms** you will build manually:

1. **Reset:** Handling the initial hardware state upon power-up.
2. **Initialize memory:** Setting up the RAM (copying ‘.data’, zeroing ‘.bss’) so C code can safely execute.
3. **Start application:** Transitioning control to the application logic.
4. **React to events & Schedule:** Handling hardware interrupts and deciding which task gets CPU time next.
5. **Synchronize:** Ensuring tasks do not corrupt shared data or peripherals.

1.4 The Target Machine: Cortex-M3 and QEMU

Embedded software is built for a known processor and a fixed memory map. It cannot rely on an OS to create a process, assign an address space, or arrange standard I/O.

1.4.1 Microcontroller (MCU) vs General-Purpose CPU

A **microcontroller (MCU)** integrates onto a single chip the components that a desktop computer splits across a motherboard.

- **Integration:** A desktop CPU is just a processing core requiring external buses for RAM and I/O. An MCU has the CPU, RAM, Flash, and Peripherals (Timers, UART, GPIO) built directly into the same chip.
- **Memory Management:** Desktop CPUs use an MMU to provide isolated virtual address spaces. An MCU relies on a *single flat physical address map* shared by code, data, and peripherals (No MMU).
- **Hardware Interaction:** On an MCU, hardware is simply memory you write to directly. There is no abstraction layer between your code and a device register.

1.4.2 The QEMU target (mps2-an385)

Instead of physical hardware, the course uses **QEMU**, a software emulator. Specifically, it targets the **mps2-an385** machine, which models an **ARM Cortex-M3** processor. The linker script strictly defines its physical memory map:

- **FLASH (Origin: 0x00000000, 4 MiB):** Non-volatile. Holds the initial image: vector table, executable instructions (‘.text’), and read-only constants (‘.rodata’).
- **SRAM (Origin: 0x20000000, 4 MiB):** Volatile. Holds mutable state: initialized globals (‘.data’), zero-initialized globals (‘.bss’), the heap, and the stack.
- **The Stack:** It starts at the very top of the RAM (0x20400000) and *grows downwards* to lower addresses.

1.4.3 Peripheral Registers and the volatile Keyword

In later weeks, device drivers will interact with peripherals by reading/writing to fixed memory addresses. These memory accesses **MUST** be declared **volatile** in C.

- **Why volatile?:** It tells the compiler that the value at that memory address can change outside the program’s control (e.g., a hardware sensor updating), or that a write triggers a physical effect. Without **volatile**, the compiler’s optimizer might delete the memory access, assuming it’s an unused ordinary variable.
- **Note:** **volatile** does **not** provide atomicity or concurrency protection; it only ensures the access is physically executed by the CPU.

1.5 Technical Foundations & Toolchain

1.5.1 From Source Code to a Running Program

The goal of this phase is to understand how to compile and run an application step by step, working directly with commands without relying on an IDE (like Visual Studio Code). Often, development environments hide the real compilation process behind a simple "Play" button, but when developing operating systems, it is essential to understand what actually happens under the hood.

The toolchain is a pipeline composed of several small tools, where the output of one becomes the input of the next:

- **C Preprocessor:** manages and expands directives like `#include` and `#define`.
- **Compiler:** takes the preprocessed code and generates assembly code (`.s`).
- **Assembler:** converts the assembly into machine code, producing object files (`.o`).
- **Linker:** concatenates matching sections, resolves symbols to final addresses, and through an `.ld` script, generates the final binary image (the ELF image).

1.5.2 Why Cross-Compilation?

Developing for embedded systems introduces an architectural problem: the computer we are writing the code on (Host) is different from the machine where the code will be executed (Target)[cite: 1]. The host is typically an x86-64 or Apple Silicon machine, whereas our target is an ARM Cortex-M3 processor[cite: 1].

A normal `gcc` compiler installed on the computer emits code optimized for the machine it runs on, which makes it useless for our purpose[cite: 1]. For example, a recent Mac's native compiler would generate code for a 64-bit ARM architecture, while our target board requires code for a 32-bit ARM architecture.

For this reason, a **cross-compilation toolchain** (specifically `arm-none-eabi-*`) is required[cite: 1]. It runs on the host but generates code for the target's ISA (Instruction Set Architecture)[cite: 1].

The `none-eabi` nomenclature is crucial to understand the environment we are working in:

- **none:** indicates the absence of a host operating system on the target (bare-metal development)[cite: 1].
- **eabi:** indicates the use of the standard embedded ARM ABI (Application Binary Interface)[cite: 1].

Working in a "bare-metal" environment means not having the standard libraries typical of desktop operating systems: there will be no `printf` to a console, no `malloc` dynamic allocation supported by an OS, and no `_start` function from a C library unless we explicitly provide it[cite: 1].

Checking the Toolchain: Host vs Target

To concretely verify the difference between compilers, we can simply query the toolchain from the terminal:

- By running `gcc -dumpmachine`, you get the architecture of your host machine (e.g., a 64-bit architecture for a recent desktop system)[cite: 1].
- By running `arm-none-eabi-gcc -dumpmachine`, the output will clearly indicate a 32-bit ARM architecture followed by the `none` environment.

Getting two completely different target strings for the same command typed on the same computer demonstrates the fundamental concept of a cross toolchain[cite: 1]. When writing an operating system from scratch, it is imperative to use this type of compiler, generate the binary file, transfer it to the target machine, and finally run it.

1.5.3 Host, Target, and Emulator

During the development of software for embedded systems, it is essential to keep three key roles distinct in the compilation and testing process[cite: 1]:

- **Host:** This is the PC we work on (e.g., our laptop) and where the text editor, toolchain, and other development tools are executed[cite: 1].
- **Target:** This is the physical hardware destination machine (in our case, a board with an ARM Cortex-M3 processor) for which the executable (the ELF image) is generated[cite: 1].
- **Emulator:** This is a virtual machine that simulates the target's hardware via software directly on our host[cite: 1]. In our case, lacking the physical board, we will use **QEMU** to test the execution of the programs[cite: 1].

1.5.4 Reset Code and the Transition to C

When running a program on a desktop computer, the operating system and the C runtime take care of preparing the memory and starting the program automatically[cite: 10]. In a bare-metal environment, at boot time, no such service exists[cite: 10].

To execute even just the first instruction of our `main()` function, it is necessary to have startup code (`boot/startup.c`) that acts as the bridge between the reset hardware and the application[cite: 10].

This code is responsible for making the C runtime's assumptions true before `main()` runs: it guarantees that every global variable already has its promised value (by initializing the `.data` and `.bss` memory sections) by the time `main()` starts[cite: 10].

The startup source also defines crucial bare-metal safety mechanisms:

- **Default_Handler:** An infinite loop used for unimplemented exception vectors. Stopping in that known loop is much safer and more diagnosable than falling into arbitrary memory after an unexpected hardware exception.

1.5.5 Preprocessing: Assemble the Translation Unit

When we start compiling (e.g., by typing the classic `gcc` command), the process is not monolithic but is divided into very distinct phases. The very first phase is handled by the **C preprocessor**. The preprocessor analyzes the C source code looking for directives (commands starting with the `#` character, such as `#include` or `#define`) and expands these macros before the compiler actually sees the program[cite: 1].

This is purely text substitution, without any machine code generation[cite: 1]. For example, upon encountering an `#include` command, the preprocessor locates the specified file and essentially "copies and pastes" it into the source file, replacing the original directive. The result of this operation is a *translation unit*: a single file of pure C code with all macros and headers already expanded[cite: 1].

Try It: Preprocess Only

We can instruct the toolchain to run *only* the preprocessing phase by using the `-E` flag. The command to run in the terminal is the following[cite: 1]:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding  
-E src/main.c -o build/main.i
```

In this string, we provide several options to describe to the toolchain the characteristics of the machine we actually have available[cite: 1]:

- `-mthumb` and `-mcpu=cortex-m3`: these indicate, respectively, to use the Thumb instruction set and that the target CPU is a Cortex-M3[cite: 1].
- `-nostartfiles` and `-ffreestanding`: these signal that we are working in a bare-metal environment, meaning without standard operating system libraries or default startup files[cite: 1]. These options will be the default for our projects.
- `-E`: commands the compiler to stop immediately after preprocessing[cite: 1].

If we open the generated output file (`main.i`), we will notice fundamental changes compared to the original source:

1. All text comments present in the source have been removed by the preprocessor.
2. The `#include <stdint.h>` directive has disappeared, entirely replaced by the expanded text (the typedef definitions) contained in that header file[cite: 1].
3. The original C code of the `main()` function has remained textually identical at the bottom of the file[cite: 1].

This demonstrates that the actual compiler, in subsequent steps, will never read our original `main.c`, but will work on this "translated" file (`main.i`), which can become huge if there are many files to include.

1.5.6 Compilation: C Becomes Target Instructions

Once preprocessing is finished, the compiler itself comes into play. The role of this step is to check the C syntax and translate the preprocessed code into **assembly** code specific to our target architecture[cite: 1]. All the options we provided earlier (such as `-mcpu`) are needed precisely by the compiler to know exactly which instructions to generate for 32-bit ARM microprocessors.

Compilation and Target Architecture

There are many 32-bit ARM microprocessors available, so it is necessary to specify the exact model being used, which in our case is the Cortex-M3. This specific CPU supports the Thumb instruction set (Thumb-2)[cite: 1]. Therefore, the compiler must be instructed precisely about the target CPU by providing specific options[cite: 1]. Finding the correct configuration involves matching the hardware datasheet specifications with the extensive GCC compiler documentation. Once the correct configuration is established and understood, it can be seamlessly applied to all future projects running on that same target machine.

Try It: Stop After Compilation

To observe the compilation phase without proceeding further in the pipeline, we can run the compiler using the `-S` flag (instead of the `-E` flag used for preprocessing)[cite: 1]. The `-S` option instructs the toolchain to run the preprocessor, compile the code, and stop immediately after producing the assembly version of the C program[cite: 1].

The output is typically stored in a file with a `.s` extension, such as `main.s`. Opening this file reveals that the original high-level C program has been translated into low-level ARM assembly instructions[cite: 1]. These instructions are written using human-readable mnemonics, not raw binary code, proving that the compiler's role at this specific stage is purely translation into assembly language[cite: 1]. While reading these files is not strictly required for this course, it is crucial to know that this intermediate translation step always takes place.

1.5.7 Assembly: Instructions Become Relocatable Bytes

Once the assembly version of the code is generated, the next tool in the chain is the **Assembler**. Its role is similar to a compiler, but it specifically takes the assembly code (`.s`) and translates it into binary machine code[cite: 1].

The output of this phase is an **object file** (identifiable by the `.o` extension). However, it is important to emphasize that the binary code contained in an object file is still not executable firmware[cite: 1]. Because projects are usually composed of multiple source files, the assembler processes each file in complete isolation. As a result, the generated object files contain machine-code bytes, section names, and symbol definitions, but they still lack proper, final memory addresses[cite: 1]. They may also contain unresolved references to code located in other object files (for instance, `startup.o` referring to `main`), which cannot be resolved at this stage[cite: 1].

Try It: Assemble to an Object File

To run the complete compiling chain up to the end of the assembly phase (executing preprocessing, compiling, and assembling sequentially), the `-c` flag must be used[cite: 1]. We can apply this to the different source files in our project:

- `arm-none-eabi-gcc [...] -c boot/startup.c -o build/boot/startup.o`[cite: 1]
- `arm-none-eabi-gcc [...] -c src/main.c -o build/src/main.o`[cite: 1]

By doing this, we generate the object files `startup.o` and `main.o`. Unlike the previous `.i` and `.s` files, these object files represent the first output in the pipeline that is entirely in binary format[cite: 1]. Opening them with a standard text editor will reveal unreadable characters, as they are no longer in a human-readable format.

Inspecting Object Files

To understand what is inside an object file, we can use a tool provided by our toolchain called `arm-none-eabi-nm`[cite: 1]. This tool reads the `.o` files and extracts information in a human-readable way, specifically focusing on "symbols"[cite: 1].

Symbols represent the memory addresses that our program uses, which at this assembly stage are still left open and undefined[cite: 1]. For example, if we run the tool on our object files, we can look for the `main` function:

- In `startup.o`, there is a call to the `main` function, but it is not defined there. The tool will label this symbol with a **U** (Undefined), meaning it is referenced but its address is unknown[cite: 1].
- In `main.o`, the function is actually written and defined. The tool will label this symbol with a **T** (indicating it is defined in the `.text` section)[cite: 1].

This separation is completely normal: the files are compiled in isolation, and it will be the linker's job to connect the "U" to the "T" and assign the final memory addresses[cite: 1].

1.5.8 Linking: Creating the Executable

As established, object files are not runnable firmware[cite: 1]. In ARM architecture, every assembly line corresponds to a 32-bit instruction with a fixed binary encoding. However, whenever an instruction needs to reference a memory address (like calling a function), those bits are left blank and filled with symbols instead.

To convert these isolated objects into a final executable, we need a last step: the **Linker**[cite: 1]. The linker takes all the independent object files, concatenates matching sections, resolves every symbol to a final address, and patches all the references[cite: 1]. The output is an executable file formatted as **ELF** (Executable and Linking Format), which is the standard format originally designed in Unix and used globally (unlike the proprietary `.exe` format used by Windows)[cite: 1].

The Linker Script

When compiling a program on a standard PC, the operating system manages memory automatically, so the linker already knows what to do[cite: 1]. However, in a bare-metal embedded system, there is no operating system to rely on[cite: 1].

Therefore, we must explicitly instruct the linker on how to organize the different pieces of code in memory using a **Linker Script** (usually a `.ld` file)[cite: 1]. This script specifies the physical addresses of the hardware, detailing exactly what goes into the Flash memory (e.g., `.text`, `.rodata`) and what goes into the SRAM (e.g., `.data`, `.bss`, `stack`)[cite: 1]. Without this script, the toolchain wouldn't know the chip's specific memory map[cite: 1].

Try It: Link the Objects

If we want to run the compiler without stopping at intermediate steps and link the project, the command becomes more complex because we must include the linker script and all the necessary object files[cite: 1].

The command looks like this: `arm-none-eabi-gcc [...] -T scripts/mps2_m3.ld [...] build/boot/startup.o build/src/main.o -o build/scratch-os.axf`[cite: 1]

Here, the `-T` flag is crucial because it tells the compiler which linker script to use (`scripts/mps2_m3.ld`)[cite: 1]. After running this, we obtain our final, linked ELF image (`scratch-os.axf`). If we inspect it again with `arm-none-eabi-nm`, the `main` symbol will no longer be marked as "U" (undefined), but will have a finalized memory address assigned by the linker[cite: 1].

The Linker Script and the Map File

To perform the final link, the linker needs a strict description of physical memory[cite: 14]. To provide this, the link command in the Makefile uses specific flags assigned to the `LDFLAGS` variable:

```
LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
```

- **The Linker Script (-T):** The first option chooses the course linker script (`scripts/mps2_m3.ld`)[cite: 14]. The script begins by declaring the **FLASH** and **RAM** regions, then specifies the output sections that occupy them. Its internal syntax defines how it places the vector table so the processor finds it at reset, how it splits an initialized global's storage between a Flash load address and a RAM run address, and how it reserves the heap and stack. While writing this script from scratch is a Week 2 topic, for now, it is provided as-is to make the build work.
- **The Map File (-Map):** The second option emits a map file (`build/output.map`)[cite: 14]. This file is an invaluable explanation of exactly where the linker placed each input contribution and symbol[cite: 14].

For Week 1, reading the generated map file is enough evidence that the script successfully assigned the final addresses and generated a valid executable binary file[cite: 14].

1.5.9 The ELF Image and its Map

The final target file generated by the toolchain is `build/scratch-os.axf`. The `.axf` suffix is commonly used for an ARM executable containing debug information, but the underlying file format is standard **ELF (Executable and Linkable Format)**[cite: 14, 15].

The ELF file organizes the program into several useful views[cite: 15, 16]:

- **Sections:** Serve the linker and debugger (e.g., `.text`, `.rodata`, `.data`, `.bss`)[cite: 15, 16].
- **Program Headers:** Describe memory ranges that a loader (or QEMU) places in memory[cite: 15].
- **Symbols:** Assign human-readable names to final physical addresses.

Thumb State and The Entry Point

The Week 1 build generates a 32-bit, little-endian ARM ELF executable. Running the command `arm-none-eabi-readelf -h -S build/scratch-os.axf` extracts the ELF header and reports the entry point at `0x49`[cite: 15].

However, looking at the map file, the `Reset_Handler` is located at `0x00000048`. This mismatch is not an error: the low bit in the ELF entry address (`0x49` vs `0x48`) explicitly records the **Thumb state**. They describe the exact same execution entry, but the `+1` tells the CPU to execute in Thumb-2 mode.

Sections Layout in Week 1

The generated ELF contains the following allocated sections mapping directly to the physical addresses dictated by the linker script:

<code>.isr_vector</code>	address	<code>0x00000000</code>	size	<code>0x40</code>
<code>.text</code>	address	<code>0x00000040</code>	size	<code>0xa0</code>
<code>.data</code>	address	<code>0x20000000</code>	size	<code>0x0</code>
<code>.bss</code>	address	<code>0x20000000</code>	size	<code>0x0</code>
<code>.heap</code>	address	<code>0x20000000</code>	size	<code>0x1000</code>

The map file (`build/output.map`) explicitly tracks this derivation[cite: 14]. It records the 64-byte vector table at Flash address zero, followed by the startup code (`startup.o`) at `0x40`. The application code (`main.o`) begins at `0xa4`, and the `"toolchain OK\n"` string literal is placed in `.rodata` at `0xd0`.

File Size vs RAM Usage

In this specific Week 1 snapshot, the `.data` and `.bss` sections have a size of `0x0`. This should **not** be misread as evidence that RAM initialization is optional in bare-metal systems[cite: 10, 16]. It merely indicates that this specific, tiny program has no global variables needing those regions yet[cite: 16].

To accurately verify memory usage, developers run `arm-none-eabi-size build/scratch-os.axf` [17]. For Week 1, it reports 224 bytes of text, 0 bytes of data, and 4096 bytes of `bss`[cite: 17]. Note that this 4096-byte `bss` total includes the `.heap` section, deliberately reserved in the linker layout to occupy RAM[cite: 17].

1.5.10 Debug Information for Host Tools

When an operating system is finalized and released, information about the original source code is typically stripped from the binary to reduce size and protect intellectual property. However, during development and debugging, it is highly beneficial to generate binary files that contain debugging information[cite: 48].

Specific compiler options can embed source-level debug metadata inside the ELF[cite: 48]. This does not mean the target machine understands C; it still only runs machine instructions[cite: 48]. Instead, this metadata allows host-side tools (like QEMU or a debugger) to associate a specific binary instruction address with a corresponding C file line, function, or variable[cite: 48]. This bridge between high-level C code and low-level execution makes investigating crashes possible without altering the basic boot image[cite: 48].

1.5.11 What Make Actually Does

While it is crucial to understand how to compile an embedded program manually, step by step, doing so in practice is extremely inefficient[cite: 49]. Remembering every compiler flag, every linker script option, and listing every single C file becomes impossible for complex projects (for instance, the FreeRTOS kernel consists of hundreds of C files)[cite: 49].

To automate the compilation process, we use tools like **Make**, which is the standard toolchain automation used by Linux and FreeRTOS[cite: 49]. A **Makefile** is essentially a text-based script that defines a set of rules to compile many files[cite: 49].

Targets Model Desired Outcomes

The **Makefile** works by defining **targets** and the actions required for each target[cite: 50]. A target is a named file or action that Make knows how to produce[cite: 50]. For example, if you have a target named **all**, you can simply type **make all** in the terminal, and the tool will execute all the instructions associated with that target[cite: 50]. Some targets, like **clean** or **run**, are usually actions rather than actual files, and they are declared as **.PHONY** to prevent conflicts with a same-named file[cite: 50].

Make is also powerful because it tracks **prerequisites** (the dependency graph)[cite: 50, 54]. If you specify that the linking step requires all object files to be created first, Make will search for the rule to generate those object files if they do not exist[cite: 50, 54]. It works backward, starting from the desired result and executing all the necessary preceding steps[cite: 50, 54].

Prerequisites Capture Rebuild Logic

Another key feature of Make is its ability to rebuild a target only if a prerequisite is newer than the target itself[cite: 49, 51]. For example, if you change one variable in a massive project (like the Linux kernel, which can take hours to compile), you do not want to recompile everything[cite: 49, 51]. Make identifies that only one **.c** file has changed, recompiles that specific file into an object file, and then simply re-runs the linker to update the final ELF image[cite: 49, 51].

To identify if something has changed, Make uses the last modified timestamp of the files[cite: 51]. However, this can sometimes lead to issues. If you are using version control (like Git) or syncing folders via cloud services (like OneDrive or Dropbox), file timestamps can become corrupted or mistakenly set in the future. In these cases, you might modify a file, but Make will refuse to recompile it because it believes the target is already up-to-date. If you encounter strange build behaviors, checking the timestamps of your files is an excellent troubleshooting step.

1.5.12 Variables and Compiler Flags

The Makefile uses a simple language that allows the definition of variables. Variables are incredibly useful because they centralize configuration: you define them once and use them throughout the script.

For instance, the compiler flags encode crucial information such as the target architecture, warning policies, optimization levels, include paths, and bare-metal assumptions. The Makefile's core flags make the target policy explicit:

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3
```

These are not just personal preferences, but a strict contract for the build:

- **CC**: Chooses the cross-compiler (e.g., `arm-none-eabi-gcc`). The "none" nomenclature explicitly means "No OS," which has practical consequences: there is no host-supplied `_start`, no dynamic loader, no process to return to when `main()` finishes, and no guaranteed `printf` console.
- **-mthumb -mcpu=cortex-m3**: Selects the exact instruction subset implemented by the target.
- **-nostartfiles**: This is particularly important. It prevents the toolchain's usual C startup objects from taking responsibility for the reset. It forces the system to rely on our custom `boot/startup.c` to make that responsibility visible.
- **-O0 -g3**: Disables optimizations (`-O0`) and includes maximum debug symbols (`-g3`). This favors a debuggable, source-corresponding image over an optimized one.
- **LDFLAGS**: Governs memory layout and the final link.

Build Directories Keep Artifacts Separate

In a well-structured Makefile, it is common practice to save the generated executable file and intermediate objects in a dedicated directory, often named `build`[cite: 53].

Using a build directory provides several benefits[cite: 53]:

- Source directories remain readable and uncluttered[cite: 53].
- The `clean` target can remove generated files safely without risking the deletion of source code[cite: 53].

- Different configurations can coexist in separate build folders[cite: 53].
- A fresh build tests whether the dependencies are complete[cite: 53].

It is crucial to remember never to manually edit the generated object files or ELF bytes in the build directory in an attempt to change program behavior[cite: 53]. All changes must originate from the source code.

Variables and String Manipulation in Make

Using variables in a `Makefile` is not strictly mandatory; you could write raw commands by skipping variables entirely if your script consists of just one command[cite: 52]. However, it is a highly recommended practice for clean and maintainable code. By saving compiling information (such as the compiler name, the target executable name, or the `build` directory path) into variables, you create a single point of entry[cite: 52, 53]. If, for instance, you decide to switch to another compiler, you only need to change one line, and the rest of the file will still function perfectly[cite: 52]. Variables in `Make` are essentially strings, and you can perform both variable assignment and variable concatenation[cite: 52].

`Make` also possesses a powerful language for processing strings and performing replacements, somewhat similar to regular expressions. This is particularly useful when managing source files. Instead of manually writing the compiling command for 100 different C files, you can list all the `.c` files in a `SOURCES` variable[cite: 54]. Then, you can use a short string substitution command to automatically generate the list of object files (`OBJECTS`)[cite: 54]. For example, the command can parse the `SOURCES` string, split it by slashes to remove the original directory (e.g., `boot` or `src`), prepend the `build` directory, and replace the `.c` extension with `.o`[cite: 54]. This automation is why `Make` is an indispensable tool in large projects[cite: 54].

1.5.13 Defining Targets and Rules

The fundamental unit `Make` reasons about is the **rule**. A `Makefile` is composed of targets (what you want to execute) and the exact instructions to execute them[cite: 18].

The syntax for a rule involves a target on the left of a colon, followed by its dependencies (prerequisites), and then the recipe:

```
target: prerequisite1 prerequisite2
    recipe line 1
    recipe line 2
```

- **Target:** Normally a file `Make` knows how to produce (e.g., an object file or ELF image), or an action (like `clean` or `run`)[cite: 16, 18]. Actions must be explicitly declared as `.PHONY` to prevent `Make` from looking for a file with that name[cite: 16, 18].
- **Prerequisites:** Files that must already be current before the recipe runs. `Make`'s rebuild decision compares modification times: a target is out of date (and its recipe runs) if it does not exist, or if any prerequisite's timestamp is newer than the target's[cite: 16].

- **Recipe:** The sequence of shell commands that turns the prerequisites into the target.

Crucial Syntax Rule: Every recipe line **MUST** start with a literal Tab character, not spaces. This is the single most common syntax error in a first Makefile.

Diagnosing the "Missing Separator" Error

If you indent a recipe line with four spaces instead of a Tab, Make will crash and report a cryptic error like:

```
Makefile:5: *** missing separator. Stop.
```

The "separator" Make refers to is the Tab character it required to recognize line 5 as a recipe. Finding spaces instead, it tried to parse the line as a variable assignment or a new rule, and failed.

Because four spaces and a Tab look identical in an editor, do not inspect the line by simply reading it. Render its whitespace in the terminal using `cat -et Makefile`. This command marks the end of each line with `$` and each Tab with `^I`, making the missing Tab immediately obvious. To avoid this entirely, configure your editor to never expand Tabs to spaces in Makefiles.

Variable Assignment Types and Debugging

Make variables centralize configuration, but how you assign them matters[cite: 17, 18]:

- `:=` (Simply expanded): The right-hand side is evaluated once, immediately. This is the preferred method.
- `=` (Recursively expanded): The right-hand side is re-evaluated every time the variable is used. This can be slow or cause infinite loops.
- `+=`: Appends to whatever the variable already holds.

By convention, uppercase names like `CC`, `CFLAGS`, and `LDFLAGS` are reserved for tools and flags, while lowercase names are for internal bookkeeping[cite: 17, 18].

Making a Makefile Report its Own State: A Makefile can be wrong without failing (e.g., if a substitution is subtly incorrect). To debug what variables actually expanded to, you can ask the Makefile directly using `$(info ...)` or `$(warning ...)`:

```
$(info OBJECTS = $(OBJECTS))
$(warning CFLAGS is $(CFLAGS))
```

These directives are expanded while the Makefile is being *read*, not built, and they print their values to the console regardless of the target you are building, making them excellent temporary debugging aids.

Automatic Variables

Inside a recipe, a handful of special variables refer back to the rule that triggered it, so the same pattern rule can serve every matching file without repeating filenames[cite: 18]:

- `$@`: The target of this rule[cite: 18].
- `$<`: The first prerequisite[cite: 18].
- `^`: All prerequisites, space-separated, duplicates removed[cite: 18].
- `$(dir $@)`: The directory part of `$@` (a built-in function)[cite: 18].

The Week 1 pattern rule uses exactly these to automatically create build directories and compile every source file[cite: 18]:

```
$(BUILD_DIR)/%.o: %.c
    @mkdir -p $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@
```

Pattern Rules and Substitution References

A **pattern rule** (`%.o: %.c`) uses a stem `%` that matches on both sides of the colon. Given a request to build an object file, Make matches it automatically without needing one explicit rule per file[cite: 19].

Additionally, a **substitution reference** like `$(SOURCES:%.c=$(BUILD_DIR)/%.o)` acts as a shorthand to convert the source list into the corresponding object file list.

Phony Targets

Actions like `run` and `clean` do not name actual files that Make should create[cite: 19]. Using `.PHONY` tells Make that a target is not a file, ensuring its recipe always runs when requested and preventing conflicts if a file with the same name exists[cite: 19]. `all` is the conventional default goal[cite: 19].

Reading the Week 1 Makefile as a Dependency Graph

Put together, the checkpoint's complete `Makefile` is structured as a dependency graph[cite: 20]:

```
SOURCES := boot/startup.c src/main.c
OBJECTS := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all clean run
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
    @mkdir -p $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@

$(BIN): $(OBJECTS)
```

```

$(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

run: $(BIN)
    qemu-system-arm ...

clean:
    rm -rf $(BUILD_DIR)

```

Running `make` walks this graph bottom-up[cite: 20]. This provides the core reproducibility benefit: the graph records how the image is made, and re-running it is always correct whether nothing changed, one file changed, or the build directory does not exist yet[cite: 20].

A Disciplined First Build Loop

The `Makefile` operates on a dependency graph[cite: 54]. For instance, if the target executable (`$(BIN)`) cannot be generated because the object files (`.o`) are missing, Make searches for the rule that dictates how to create them[cite: 54].

This rule typically looks like this: `$(BUILD_DIR)/%.o: %.c`[cite: 54]

This means that for every `.o` file on the left side, the dependency is the corresponding `.c` file on the right side[cite: 54]. If the `.o` file is missing or outdated, this rule is triggered[cite: 54]. The actions associated with this rule usually involve:

1. Creating the `build` folder if it does not already exist.
2. Running the compiler with the `-c` flag, which instructs it to compile and stop before linking[cite: 54].

When you simply type `make`, the tool executes these rules in reverse order: it checks for the final ELF file, realizes it needs the object files, realizes it needs to compile the source files, and then executes the steps sequentially from the bottom up. This provides a highly disciplined build loop: you can run `make` to build the ELF, inspect the terminal for any compiler errors, run the generated ELF, and compare the observed output with what you expect[cite: 55].

1.5.14 QEMU Runs a Board Model

The final crucial target in our `Makefile` is the `run` command[cite: 54]. The rule for `run` depends on `$(BIN)`: the finalized ELF file must be available before execution can begin[cite: 54]. If the file is present, Make executes QEMU (`qemu-system-arm`)[cite: 56].

The emulator requires several parameters to function correctly:

- `-machine mps2-an385`: This tells QEMU which specific hardware board it needs to emulate, ensuring the CPU and memory map match our Cortex-M3 target[cite: 56].
- `-kernel`: This parameter passes the generated ELF file to the emulator[cite: 56].

Other options might include disabling the graphical screen (since we only need the terminal output) and enabling semi-hosting to emulate a debugging cable attached to the board.

When this command runs, QEMU emulates the entire physical process: it acts as if a program has flashed the ELF into memory, the power is turned on, and the board starts executing[cite: 56].

1.5.15 QEMU, Semihosting, and the Observed Result

The `run` target first ensures the ELF exists, then invokes QEMU with specific parameters: `-machine mps2-an385`, `-kernel build/scratch-os.axf`, `-nographic`, and semihosting enabled. QEMU supplies the target-side machine model while presenting its console directly through the host terminal.

Why Avoid `printf`?

The application in `src/main.c` intentionally avoids standard `printf`. The custom reset path does not perform the complete C-library initialization expected by newlib's semihosting-backed `stdio`.

Instead, `semihost_write0` makes the ABI-level request directly:

1. Places semihosting operation `0x04` (`SYS_WRITE0`) in register `r0`.
2. Places the NUL-terminated message pointer in register `r1`.
3. Executes the Thumb breakpoint instruction `bkpt 0xab`.

With semihosting enabled, QEMU intercepts the breakpoint trap, writes the requested string to the host console, and resumes target execution.

Running the Demonstration

To execute the demonstration from the checkpoint directory:

```
make
make run
```

The observable terminal output is:

```
toolchain OK
```

Interpreting the Output

After printing, `main()` returns to the `Reset_Handler`, which enters its final infinite spin loop[cite: 21]. QEMU continues running because the target continues executing that loop; it must be interrupted manually[cite: 21].

Therefore, the single line `toolchain OK` is **not** an exit notification. It is concrete, empirical evidence that the image successfully reached the intended point in the boot path[cite: 21].

The Final Spin Loop and QEMU Execution

It is important to note what happens after the `main()` function finishes its execution. `main()` itself never returns to an operating system because, in a bare-metal environment, there is no OS to return to[cite: 21].

When it does return, the `Reset_Handler` enters a final `for(;;) {}` spin loop, because a bare-metal image has nowhere else to go[cite: 21]. This ensures the processor remains safely locked in a known state instead of wandering into uninitialized memory and executing random garbage data[cite: 21].

Crucial Note on Emulator Behavior: QEMU keeps running that infinite loop; it does not exit on its own. Any explanation saying that QEMU automatically exits after printing the final message is strictly incorrect for this checkpoint.

1.6 A Practical Inspection Routine

When working on this checkpoint, it is essential to inspect the source and generated evidence in the exact order of the pipeline:

1. **Read the Makefile** to see the commands and target flags.
2. **Read the linker script** (`mps2_m3.ld`) to establish legal memory regions and section order.
3. **Read `startup.c`** to see how linker symbols become initialization loops and how reset reaches C.
4. **Read `main.c`** to identify the expected observable behavior.
5. **Use `map` and `ELF` tools** to verify that the linker implemented the intended layout:

```
arm-none-eabi-readelf -h -S build/scratch-os.axf
arm-none-eabi-size build/scratch-os.axf
```

1.6.1 Systematic Troubleshooting

When faults occur during development, follow a systematic troubleshooting path:

- **If the compiler rejects a command:** Check the cross-toolchain installation and the Makefile flags.
- **If a symbol is unresolved:** Check the object list and matching declarations.
- **If a section overflows:** Inspect the linker-script memory regions and the map file.
- **If QEMU starts but the expected output is absent:** Check the reset path, the call to `semihost_write0`, and the semihosting options.

1.6.2 Foundational Significance

This checkpoint serves as the permanent foundation for all subsequent work: future peripherals will use the same physical-address model, task code will still enter through the same reset path, and every future image will be linked, inspected, and booted through this exact toolchain. Week 1 makes these relationships explicit before later operating system abstractions obscure them.

Chapter 2

Boot and CPU Configuration

2.1 Introduction to Low-Level CPU Control

In the previous phase, we built a tiny program for a Cortex-M3, ran it in QEMU, and saw it print one line. Several pieces of that program were handed to us as-is: the linker script and the startup code. The primary goal of this phase is to stop trusting those pre-built components, move beyond simply using the toolchain, and take direct control of the CPU[cite: 25].

By the end of this phase, developers will understand every instruction the CPU executes between power-on and the first line of `main()`[cite: 25]. The development process is structured around three core units:

- **Where code and data live:** Understanding the memory layout and directing the linker script to place code and data in the correct physical locations[cite: 25].
- **Taking control of the CPU:** Controlling the boot process through an assembly startup script, configuring faults, and using a debugger[cite: 25].
- **Life without an OS underneath:** Operating in a freestanding environment to comprehend bare-metal limitations, splitting the program into privileged and unprivileged parts, and building a system call (`svc`)[cite: 25].

2.1.1 Where You Start and Where You End

All the work in this phase begins on a stripped-down starting skeleton. The original C startup code has been removed. In its place is a new, almost empty assembly file (`boot/startup.s`). It contains only the two words the CPU absolutely needs at reset (the stack address and the PC), and a `Reset_Handler` that does nothing except call `main()` and loop forever.

This minimal setup exposes a critical physical reality. If we try to run a program that prints a standard string literal and an initialized global character array (which lives in the `.data` section), the output will be incomplete:

```
make
make run
```

```
main: hello from the base
```

The second line is missing. Nothing crashes, and no error is printed: the second call simply prints an empty string. Understanding exactly why this happens—and writing the assembly code to fix it—is the primary objective of this chapter.

Once the boot code, memory initialization, and fault handlers are correctly implemented from scratch, the final program will successfully print:

```
main: privileged hello
main: this text lives in .data
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

Note: As established previously, QEMU does not exit automatically when the program finishes because the bare-metal image enters an infinite spin loop.

2.2 Memory Technologies in Embedded Systems

When designing an operating system or bare-metal software, it is crucial to understand the different memory technologies available on the hardware, as this dictates where instructions and variables are placed.

2.2.1 SRAM (Static RAM): The Working Memory

- SRAM sits directly on the CPU's memory bus, meaning every byte has a specific address and can be accessed using standard load/store instructions[cite: 66].
- It is extremely fast; on a microcontroller, a single access typically takes about one clock cycle[cite: 66].
- It is byte-writable, allowing the CPU to change single bytes at any time[cite: 66].
- It is volatile, meaning all data is lost when power is removed, and its contents are entirely undefined (random bits, not zeros) at power-on[cite: 66].
- Due to its writable nature, SRAM is the designated location for anything that changes during program execution, such as variables and the stack[cite: 66, 69].

2.2.2 NOR Flash: Execute-in-Place Memory

- Like SRAM, NOR Flash sits on the memory bus and can be addressed byte by byte, allowing fast random-access reads[cite: 67].
- Because it is on the address bus, the CPU can fetch and execute instructions directly from it without needing to copy the program to RAM first[cite: 67]. This feature is known as execute-in-place (XIP)[cite: 67].
- It is non-volatile, ensuring the program remains intact after power-off[cite: 67].
- Writing to NOR Flash is complex and cannot be done with simple store instructions[cite: 67]. It requires first erasing an entire sector (turning all bits to 1) and then programming specific words (turning bits from 1 to 0), a process that takes microseconds to milliseconds[cite: 67].

- Consequently, at runtime, NOR Flash is effectively treated as read-only memory[cite: 67]. It is used to store the code itself and read-only constants (like string literals)[cite: 69].

2.2.3 NAND Flash and Block Devices

- NAND Flash is the technology used in SSDs, hard disks, and SD cards[cite: 68].
- Unlike NOR Flash, it does not sit on the CPU's memory bus and is instead accessed through a separate controller[cite: 68].
- Data cannot be addressed byte by byte; it must be accessed in large blocks or pages (e.g., 512 bytes to 4 KB)[cite: 68].
- Because it lacks direct memory addressing, the CPU cannot fetch instructions from NAND Flash, nor can it read single variables directly[cite: 68].
- To execute a program stored on a NAND device, an OS loader must first copy the blocks into RAM, which is the standard process for desktop computers[cite: 68]. Microcontrollers generally skip this step by running their programs directly from NOR Flash[cite: 68].

2.2.4 Applying Constraints to C Code

Given the physical characteristics of these different memory technologies, an operating system must correctly place every element of a C program based on its requirements[cite: 5]. The following table summarizes the hardware constraints:

Technology	On the bus?	Execute from it?	Write at run time?	Keeps data?
SRAM	yes	yes	yes, bytes	no
NOR flash	yes	yes (XIP)	no (erase/program)	yes
NAND / disk	no (blocks)	no	through a driver	yes

- **Code and constants:** Instructions and read-only data (like string literals) should remain in NOR Flash and be executed or read directly from there[cite: 5].
- **Variables (uninitialized):** Standard uninitialized variables that need to be updated at runtime must live in SRAM[cite: 6].
- **Variables (initialized):** Variables defined with an initial value (e.g., `int x = 5;`) present a unique challenge. The variable lives in RAM, but RAM is random at power-on. Therefore, the initial value 5 is stored non-volatily in NOR Flash, and something must explicitly copy it into RAM at boot, before `main()` runs[cite: 6].

Crucial Note: This requirement to manually copy initialized data is the exact reason why the base program from the previous section prints only one line and fails to print the `.data` string!

2.2.5 The Cortex-M3 Memory Map

To tell the compiler where to place these different elements, a developer must first thoroughly understand the hardware's memory map. On a 32-bit Cortex-M microprocessor, the CPU sees a single 4 GB physical address space[cite: 6]. The regions within this space are fixed by the ARM architecture, while the chip manufacturer decides how much real memory to put in each region[cite: 6]:

Address range	Region	On mps2-an385
0x0000_0000–0x1FFF_FFFF	Code	4 MB at 0x0: the program ("FLASH")[cite: 6]
0x2000_0000–0x3FFF_FFFF	SRAM	4 MB at 0x2000_0000: "RAM"[cite: 6]
0x4000_0000–0x5FFF_FFFF	Peripherals	UART, timers, ... [cite: 6]
0x6000_0000–0xDFFF_FFFF	External	Not used here[cite: 6]
0xE000_0000–0xE00F_FFFF	CPU registers	Control of the CPU itself[cite: 6]

Even though we are emulating this hardware in QEMU (where the 4 MB at address 0 is technically just fast RAM pre-loaded with the program image), the emulator enforces these access rules strictly. We treat it as read-only, exactly as we would on a real microcontroller with real flash[cite: 6].

2.2.6 The Linker Script: Mapping the Chip

The **Linker Script** (e.g., `scripts/mps2_m3.ld`) is the file where we describe the hardware's memory map to the toolchain and provide instructions on how to place our code within it[cite: 71].

The script is divided into core sections[cite: 71]:

- **MEMORY**: Defines which physical memory regions exist, their start addresses, and their sizes[cite: 71].
- **SECTIONS**: Instructs the linker on which input sections from the `.o` files should go into which output regions[cite: 71].
- **ENTRY**: Defines where execution starts[cite: 71].

Defining the Memory Regions

The **MEMORY** block dictates the layout of the available hardware. For example[cite: 72]:

```
MEMORY {
    FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 4M
    RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 4M
}
```

Each region declaration includes an identifier (e.g., `FLASH`), its starting address (`ORIGIN`), and its total size (`LENGTH`)[cite: 72]. The linker uses this information to verify that the compiled code fits; if the data assigned to a region exceeds its `LENGTH`, the link process will fail[cite: 72].

The letters in parentheses specify the access rights for that region[cite: 72]:

- **r**: Read

- **w**: Write
- **x**: Execute

For instance, **(rx)** indicates that the **FLASH** region can be read and executed, but not written to during normal runtime[cite: 72].

Defining the Memory Regions (Continued)

In the **MEMORY** block, we specify the initial parameters of each physical memory region based on the hardware constraints:

- We define the starting address of the **FLASH** region at **0x00000000**, which is the predefined start of the code section for Cortex-M3 processors[cite: 72].
- We define the starting address of the **RAM** region immediately after, at **0x20000000**, which is the designated space for static RAM on this architecture[cite: 72].

Every address is written in hexadecimal using 32 bits (8 characters). It is imperative that this description perfectly matches the physical hardware specifications. If a developer erroneously marks a region as writable (**w**) when the underlying technology is strictly read-only (like NOR Flash), the linker will silently generate the ELF file without raising errors. However, the program will inevitably crash at runtime when it attempts an illegal operation[cite: 72].

Symbol Assignments for the Stack Pointer

Within the linker script, we can define symbols that the linker will substitute with exact memory addresses during the linking process. A crucial example is calculating the starting address of the stack[cite: 72].

On ARM microprocessors, the stack grows downwards (backwards in memory)[cite: 72, 98]. Therefore, the most efficient approach is to set the initial stack pointer to the very end of the RAM[cite: 72]. In the script, this is done by defining a symbol (e.g., **_estack**) and assigning it the value of the RAM's starting address plus its total length[cite: 72]:

```
_estack = ORIGIN(RAM) + LENGTH(RAM);
```

Organizing Output Sections

The core function of the linker script is to declare which sections should exist in the final ELF image and what content should be placed inside them[cite: 73].

When compiling a C program composed of multiple source files, the compiler produces multiple object files (**.o**), each divided into various input sections. The linker script uses rules to aggregate these scattered input sections into unified output sections[cite: 73].

For example, to organize the executable code, we define a **.text** output section[cite: 73]:

```
.text : {  
    *(.text*)  
    *(.rodata*)
```

```

    _etext = .
} > FLASH

```

- `*(.text*)`: This command tells the linker to examine every input object file (`*`), find any section whose name starts with `.text`, and place its contents here[cite: 73]. The wildcard ensures that even if sections are slightly renamed by the compiler, they are correctly included[cite: 73].
- `*(.rodata*)`: This instructs the linker to take all read-only constants from all object files and append them immediately after the code within this same output section[cite: 73].
- `_etext = .`: The dot (`.`) represents the location counter (the current memory address)[cite: 73]. This line assigns the current address (which corresponds to the end of the `.text` and `.rodata` data) to a symbol named `_etext`[cite: 73]. This provides a programmatic way to know exactly where the code section finishes in memory[cite: 73].
- `> FLASH`: Finally, this specifies that the entire `.text` output section must be placed physically in the region labeled `FLASH` defined in the `MEMORY` block[cite: 73].

2.2.7 Run Address and Load Address: The `.data` Section

For read-only data and executable code (the `.text` section), memory placement is straightforward because they exclusively reside in Flash memory. Code runs exactly where it is stored.

However, writable global variables that are initialized (e.g., `int x = 5;`) require a different approach. The `.data` section needs two distinct addresses:

```

_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;
    *(.data*)
    _edata = .;
} > RAM AT > FLASH

```

Every output section in the ELF file has two addresses:

- **VMA (Virtual Memory Address / Run Address)**: This is where the program *uses* the section when it runs. For variables, this must be in RAM so their values can be updated.
- **LMA (Load Memory Address)**: This is where the section's bytes are permanently *stored* in the final binary image, which must be in FLASH.

The line `> RAM AT > FLASH` explicitly dictates this split.

The Linker Does Not Copy Data

The linker script also records the addresses the boot code will need: `_sdata` and `_edata` define the start and end of `.data` in RAM, while `_sidata` points to the flash address where the stored copy of the initial values begins.

The key point is that **nothing moves the bytes from the LMA to the VMA by itself**. The linker only records the two addresses, and the CPU certainly does not copy anything at reset. **Your boot code must manually perform the copy.**

Here is the resulting memory layout mapped by the script:

```
FLASH: [ .isr_vector ] [ .text + .rodata ] [ copy of .data ] <- _sidata
RAM:   [ .data ] [ .bss ] [ heap ] . . . . . [ stack v ] <- _estack
      ^ _sdata ^ _sbss
```

2.2.8 BSS, Heap, and Stack Layout

Other sections of the program must also be defined in the linker script to properly manage the RAM:

- **.bss Section:** This section contains uninitialized global variables. Unlike the `.data` section, it has no LMA content in Flash; the image only records its size[cite: 75]. It is assigned exclusively to RAM, and the boot code is responsible for filling this memory area with zeros using the `_sbss` and `_ebss` symbols[cite: 75].
- **Heap and Stack:** If the program utilizes dynamic memory allocation, a heap region must be defined. In standard embedded layouts, the heap grows upwards from the end of the `.bss` section, while the stack grows downwards from the very end of the RAM (`_estack`)[cite: 75].

Safety Asserts in the Linker Script

To ensure system stability, the linker script can include logical conditions using the `ASSERT` command[cite: 75]. For instance, an `ASSERT` can be programmed to verify that the predefined limits of the stack (`_StackLimit`) and the top of the heap (`_heap_top`) do not overlap[cite: 75]. If this mathematical condition is violated during compilation, the linker will immediately abort the process and throw an error, preventing a catastrophic memory corruption at runtime[cite: 75].

2.2.9 The Role of Linker-Defined Symbols

Symbols defined in the linker script such as `_sdata`, `_edata`, `_sidata`, `_sbss`, `_ebss`, and `_estack` are unusual: they have **no storage**[cite: 9, 10]. There is no memory cell called `_sdata` holding a value. The linker only gives each of them an **address**, and the address itself *is* the information.

This distinction is critical when using them from C:

```
extern uint32_t _sdata, _edata; /* "there is something at this address"
uint32_t *start = &_amp;sdata; /* OK: the address (0x20000000)
*/
uint32_t oops = _sdata; /* wrong idea: reads RAM at that address
```

You declare them as `extern` objects only so that C lets you take their address with `&`. Reading `_sdata` itself would read whatever happens to be in RAM at `0x20000000`, which is not what you want. In assembly, the same idea is written `ldr r0, =_sdata`, which means `r0 = &_sdata`; in C. The boot code uses these symbols as loop bounds to copy data and zero out the BSS[cite: 9, 10].

Memory Alignment

Another critical instruction often found in linker scripts is the `ALIGN` command. Many RISC processors require instructions and data to be aligned in memory (typically in 4-byte words). If a section ends at an unaligned address, `ALIGN(4)` inserts padding. Failing to align memory properly will cause the CPU to crash.

2.2.10 Reading the Compilation Results

You now know what the linker script *asks* for. You can check what the linker actually *did* using three tools[cite: 10].

- **The Map File** (`output.map`): Generated by passing `-Map=` in the `Makefile`. It repeats the `MEMORY` regions and lists every output section, the input sections inside them, and the load address for `.data`[cite: 10].
- **nm Tool**: Running `arm-none-eabi-nm -n` lists all symbols sorted by address[cite: 10]. The letter before each name indicates its kind: `T/t` for text/code, `R` for read-only data, `D` for initialized data, `B` for BSS, and `A` for absolute symbols[cite: 10].
- **objdump Tool**: Running `arm-none-eabi-objdump -h` prints one line per output section, showing its size, VMA, and LMA[cite: 10].

Analyzing the Base Image

Run all three on the base skeleton:

```
make
arm-none-eabi-objdump -h build/scratch-os.axf
arm-none-eabi-nm -n build/scratch-os.axf
```

In the `objdump -h` output, you find `.isr_vector` at address `0x0` with size 8 bytes, and `.text` starting right after it at `0x8`. There is no `.rodata` line because it was folded into `.text`[cite: 10, 11].

The `.data` line is the most interesting: its VMA is `0x20000000` (start of RAM), but its LMA is a small flash address (`0x60` in the base image), right after the code[cite: 10, 11]. In the `nm` output, `_sdata` has exactly that value, `0x60`. You also find `_sdata` at `0x20000000`, `_edata` 32 bytes later (the size of the string literal), `_sbss` and `_ebss` equal (no zero-initialized variables in the base), and `_estack` at `0x20400000`.

2.2.11 Which Section Does a Declaration Go To?

The compiler decides the *input* section of each thing you declare, and the linker script then decides where that section goes. For declarations at file scope (outside any function), the rules are:

Global declaration (outside any function)	Section	Region
a function body	<code>.text</code>	FLASH
<code>const int k = 3;</code>	<code>.rodata</code>	FLASH
"a string literal"	<code>.rodata</code>	FLASH
<code>int x = 5;</code>	<code>.data</code>	RAM, initial value in FLASH
<code>int y;</code> or <code>int z = 0;</code>	<code>.bss</code>	RAM, zeroed at boot

Note the last row: a variable explicitly initialized to zero also goes to `.bss`. Because "start at zero" is exactly what `.bss` guarantees, there is no reason to store the zero in Flash[cite: 11].

Pointers, Local, and Static Variables

Pointers deserve a moment of care, because a pointer is itself a variable[cite: 11]. In `const char *greeting = "hi";`, the characters "hi" form a string literal and go to `.rodata`[cite: 11]. However, `greeting`, the variable holding their address, is an initialized writable variable and goes to `.data`[cite: 11].

The section rules discussed above apply only to declarations made **outside any function**. A local variable inside a function has no section at all: it lives dynamically on the stack while the function runs, and it is gone as soon as the function returns[cite: 11].

By contrast, a **static** local variable (e.g., `static int s = 3;` inside `main`) behaves like a global variable. It keeps one fixed address for the whole run of the program, exactly like a global—it is only its *name* that stays private to the function. In the `nm` output, the compiler appends a suffix (like `s.0`) to distinguish it from other statics with the same name.

Try It: Analyzing the Output and Little-Endianness

You can test the section rules by adding these global variables to `src/main.c`, right after `data_message` (before `main`):

```
const int limit = 100;
int counter = 5;
int total;
int zero = 0;
const char *greeting = "hi";
char buffer[64];
```

Running `arm-none-eabi-nm -n build/scratch-os.axf` reveals their placement[cite: 11]. `counter` and `greeting` are marked `D` (initialized data), while `total`, `zero`, and `buffer` are marked `B` (bss)[cite: 11].

Interestingly, in the intermediate object file (`main.o`), `limit` is marked `R` (read-only data), but in the final ELF image, it is marked `T` (text)[cite: 11, 12]. This

happens because the linker script folds `.rodata` inside the `.text` output section, and `nm` classifies a symbol by its final output section[cite: 12].

To find the string literal `"hi"`, which lacks a symbol name, dump the raw contents of the `.text` output section[cite: 12]:

```
arm-none-eabi-objdump -s -j .text build/scratch-os.axf
```

At address `0x44` you find `64 00 00 00` (the value 100 for `limit`), and at `0x48` you find `68 69 00` (the characters `'h'`, `'i'`, and the null terminator)[cite: 12]. The string `"hi"` starts exactly at `0x48` in FLASH.

Pointers in RAM and Little-Endianness How does the program find the string? Through `greeting`, the pointer stored in RAM. Dump the `.data` section:

```
arm-none-eabi-objdump -s -j .data build/scratch-os.axf
```

Following the `data_message` string, you will find `counter` (represented as `05 00 00 00`) and `greeting` (represented as `48 00 00 00`).

To read these numbers, you must know one crucial hardware fact: the Cortex-M3 is **little-endian**, meaning a 32-bit number is stored with its lowest byte first. Therefore, the bytes `48 00 00 00` represent the hexadecimal number `0x00000048`. This is exactly the FLASH address where `"hi"` starts. This demonstrates that a pointer is simply a variable whose value is a memory address.

2.2.12 Advanced Memory Layout: Custom Sections and Orphans

You are not limited to the sections the compiler chooses. GCC lets you name the input section of a variable or a function yourself using compiler attributes[cite: 12]:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

Where that section ends up is up to the linker script, **if the script names it**[cite: 12]. A section that no rule in the `SECTIONS` block matches is called an **orphan**[cite: 12]. The linker does not reject it; it places it by guessing based on its flags (e.g., writable vs. executable) and putting it next to sections that look similar.

While the guess usually links fine, it places the data outside your control and outside any symbol range you defined. If you add the `tagged` variable to the base project and search for it:

```
arm-none-eabi-nm -n build/scratch-os.axf | grep -E '_sdata|_edata|tagged'
```

The output prints the linker symbols next to your variable:

```
20000000 D _sdata
20000020 D _edata
20000020 D tagged
```

Notice that `tagged` is placed at `0x20000020`, exactly where `_edata` ends. Because it is an orphan, it falls outside the `_sdata` to `_edata` loop bounds, meaning the boot code will never copy its initial value into RAM!

A new output section `.mysection` appears. The linker saw a writable, initialized section, so it placed it in RAM **right after** `.data`, and gave it its own load address in FLASH.

Now compare its address with `_edata`: `tagged` sits exactly at `_edata`, that is, *after* the end of the range `_sdata..._edata`. A boot loop that copies only `_sdata..._edata` will never copy it, so `tagged` will not hold 42 when `main()` runs. The program links without a single warning and is still wrong. That is the danger of orphans.

If you declare `const int` in a section called `.myconst` instead, the linker sees a read-only section and places it in FLASH, between `.isr_vector` and `.text`.

Forcing Variables into Specific Sections

You have the ability to force the compiler to place a variable into a specifically named section directly from your C code. This is done using compiler attributes. For instance:

```
int tagged __attribute__((section(".mySeg"))) = 42;
```

This command defines an integer variable named `tagged` and forces the linker to store it in a custom section named `.mySeg`. In the linker script, you can then define exactly where this `.mySeg` section should be placed physically, giving you absolute control over the memory layout. Even if you do not explicitly define `.mySeg` in the linker script, the linker will still attempt to manage it as an "orphan" section, though it's best practice to define it explicitly.

2.2.13 Overflowing a Region

The `LENGTH` values in the `MEMORY` block are not decoration: the linker enforces them. First, measure how much flash the base image needs:

```
arm-none-eabi-size build/scratch-os.axf
```

FLASH has to hold `text + data`: the code and constants, plus the stored copy of `.data`'s initial values. For the base skeleton, this is $96 + 32 = 128$ bytes.

If you edit `scripts/mps2_m3.ld` and set `FLASH LENGTH = 100`, you can test the overflow protection. Rebuild with `make -B` (because `make` does not track linker script changes automatically, `-B` forces everything to be rebuilt)[cite: 12]. The link fails with:

```
region 'FLASH' overflowed by 28 bytes
```

and $128 - 100$ is exactly 28[cite: 12]. Setting `LENGTH = 128` allows the link to succeed because the image fits exactly.

If you restore FLASH and set `RAM LENGTH = 8K` instead, `_estack` moves down to `0x20002000`. The 4 KB stack reservation starts 4 KB below it, and the 4 KB heap no longer fits between `.bss` and the stack[cite: 12]. The failure comes from the script's own `ASSERT` line, which turns a memory corruption that would normally appear at run time into a clear error at build time[cite: 12].

2.2.14 Reset: The Hardware Contract

Once the memory layout is finalized and the ELF file is loaded into memory, the machine can be powered on[cite: 88]. Operating systems on standard desktop computers rely on a BIOS, a loader, and a C runtime to initialize the environment.

However, on a bare-metal ARM Cortex-M microprocessor, absolutely none of these services exist[cite: 88].

When the Cortex-M processor comes out of reset, the hardware executes a strictly predefined and immutable sequence[cite: 88]. Before any user code runs, the hardware performs exactly two memory reads at the very beginning of the address space (0x00000000)[cite: 88]:

1. **Word 0:** It reads the first 4 bytes at address 0x0 and copies this value directly into the Stack Pointer (SP) register[cite: 88].
2. **Word 1:** It reads the next 4 bytes at address 0x4 and copies this value into the Program Counter (PC) register, jumping to that address to begin execution[cite: 88].

This behavior is a hardwired contract[cite: 88]. The CPU blindly trusts that the developer has placed the correct initial stack address and the correct starting address of the application code into these first eight bytes of memory[cite: 88]. Everything else—copying the `.data` section to RAM, zeroing out the `.bss` section, and setting up the C environment—must be explicitly handled by the developer's boot code starting at the address provided in Word 1[cite: 88].

2.2.15 The Vector Table: Controlling the Boot Process

Those two initial words (SP and PC) are the beginning of the **vector table**: a table of addresses at a fixed place in memory (address 0x0 at reset)[cite: 13].

The table exists because of **exceptions**[cite: 13]. An exception is an event that makes the CPU stop what it is doing and run a specific piece of code called its **handler**[cite: 13]. Reset is one exception; a hardware fault is another[cite: 13]. Each exception has a number, and slot *N* of the vector table holds the address of the handler for exception *N*[cite: 13]. Slot 0 is special: it is not the address of any code, but the initial value of the SP[cite: 13].

Implementing the Vector Table in Assembly

In the base skeleton, the table is written in `boot/startup.s` inside a section called `.isr_vector`, and initially has only two slots[cite: 13]:

```
isr_vector:
    .word _estack                /* 0: initial SP   (C: &_estack)
*/
    .word Reset_Handler        /* 1: reset       (C: Reset_Handler) */
```

`.word X` simply allocates a 32-bit word and stores the value `X` inside it[cite: 14, 19]. The label `isr_vector` is the address of the first word. Because the linker script places the `.isr_vector` section first in FLASH, the table lands exactly at address 0x0[cite: 13].

The Thumb Bit Constraint

When inspecting the boot sequence in a debugger, the value stored in word 1 is not exactly the raw address of `Reset_Handler`, but that address with its lowest bit set

to 1. Because instructions always sit at even memory addresses (2-byte or 4-byte aligned), bit 0 of a code address is otherwise unused. ARM uses bit 0 to indicate "this is Thumb code". The Cortex-M executes only the Thumb instruction set, so it strictly requires this bit to be 1[cite: 17, 19].

KEEP and Garbage Collection

The linker has an optional feature, enabled with `-Wl,-gc-sections`, called **garbage collection of sections**. Starting from the `ENTRY` symbol, the linker follows every reference from one section to another, and **drops every input section that nothing references**[cite: 14]. This is useful for trimming unused code, but dangerous for data that only the hardware reads: no code references the vector table, so the garbage collector would consider it useless and throw it away[cite: 14].

`KEEP(...)` in the linker script marks input sections as **always kept**, whatever the garbage collector thinks[cite: 14]. The rule `KEEP(*(.isr_vector))` exists for exactly this reason.

If you remove `KEEP` and enable garbage collection, `objdump -h` will show `.isr_vector` with size 0. Running the program will fail with a fatal `Lockup` error. The lockup makes perfect sense: at reset, the CPU still reads words 0 and 1, but those words are now the first instructions of `Reset_Handler`, not a stack address and a code address. The CPU jumps into nonsense and stops.

2.2.16 Back to the Base: Why Only One Line Prints

You now have everything you need to explain the base skeleton's behavior from the beginning of the chapter. `main()` prints two strings:

1. The first is a string literal: it lives in `.rodata`, inside `.text`, in FLASH, and the CPU reads it directly from there. It prints correctly.
2. The second is `data_message`, an initialized global array. Its VMA is at `0x20000000` in RAM, but its initial bytes are stored at a flash LMA.

`main()` reads the string at its VMA, in RAM. But in the base skeleton, `Reset_Handler` contains only `bl main` and a loop: **no code moves the bytes from the LMA to the VMA**. The RAM at `0x20000000` still holds whatever it held at power-on, which in QEMU is all zeros. A string whose first byte is zero is the empty string, so the second call prints nothing! Writing that missing copy is the goal of the next unit.

2.3 Taking Control of the CPU

2.3.1 Startup Code: Assembly vs. C

The previous sections established how to correctly place code and data into memory. The next critical phase is understanding precisely what code must be placed there to initialize the system and take full control of the CPU. The low-level startup code is essential for bridging the gap between hardware power-on and the execution of the main C program.

While this startup sequence can technically be written in C, it is highly instructive—and often more straightforward—to write it directly in Assembly language[cite: 93]. Writing this sequence in C requires complex abstractions and pointer arithmetic to manipulate low-level hardware states[cite: 93]. Assembly, conversely, provides a transparent 1:1 mapping between the instruction written and the physical action taken by the CPU, making the core initialization steps much clearer to understand[cite: 93].

2.3.2 What is a Debugger?

In this unit, you rebuild `Reset_Handler` in assembly one step at a time. To do that, you need a debugger to *see* what the CPU does[cite: 14]. A **debugger** is a program that controls another program while it runs. It can **stop** the program at any instruction (a **breakpoint**), **step** forward one machine instruction at a time, **inspect** CPU registers and memory, and **continue** execution[cite: 14].

On a desktop, a crashing program prints an error message. On a microcontroller, there is often no console at all, and a crash simply looks like "nothing happens." A debugger is frequently the *only* way to see what the CPU is really doing[cite: 14]. The standard tool is **gdb**, the GNU debugger. The ARM version is `arm-none-eabi-gdb`[cite: 15].

2.3.3 A Remote Debugger: Two Programs, Two Terminals

In this setup, the program being debugged does not run on the same "machine" as GDB[cite: 14]. It runs inside QEMU, which emulates the Cortex-M3 board. GDB talks to QEMU over a network connection (a TCP socket). This requires two separate terminals[cite: 14, 15]:

<pre>Terminal 1: make debug +-----+ QEMU: CPU + Flash + RAM TCP halted at reset <-----> gdb server on port 1234 :1234 +-----+ program output shows here</pre>	<pre>Terminal 2: arm-none-eabi-gdb build/scratch-o +-----+ gdb: knows your symbols (main, Reset_Handler...) from the .axf file +-----+ you type commands here</pre>
---	--

- **Terminal 1 (The Target):** You start QEMU with `make debug`[cite: 15]. The `debug` target in the `Makefile` runs QEMU with two crucial options: `-S`, which freezes the emulated CPU before its very first instruction, and `-gdb tcp::1234`, which starts a GDB server waiting for a connection on port 1234[cite: 15]. Anything the program prints still appears in this terminal.
- **Terminal 2 (The Host):** You start GDB and pass the `.axf` ELF file so the debugger has access to all the symbol names[cite: 15]. You type your commands here.

GDB: The GNU Debugger

The standard tool for this task is **GDB** (the GNU Debugger)[cite: 94]. While it lacks a graphical user interface and operates entirely via the command line, it

is exceptionally powerful and specifically designed for this client-server architecture[cite: 94]. To use it with our ARM target, we run the cross-compiled version (`arm-none-eabi-gdb`) and load our ELF file (`scratch-os.axf`) so the debugger has access to all the symbol names (like `main` or `Reset_Handler`)[cite: 95].

We then connect to the running emulator using the command `target remote :1234`. From there, GDB allows us to inspect CPU registers (`info registers`), examine raw memory addresses in hexadecimal (`x/wx`), step through the program one single machine instruction at a time (`stepi`), and set breakpoints to pause execution at specific labels (`break`).

Setting Up the Debugging Session

To use the debugger in practice, we must coordinate both ends of the connection:

1. Preparing the Target (QEMU): Instead of running the emulator normally, we must instruct it to start in debug mode. In the `Makefile`, there is a specific target named `debug` which launches QEMU with additional flags:

```
-s -gdb tcp::1234
```

The `-s` flag is critical: it commands the emulator to freeze the CPU immediately at reset, before the very first instruction is executed[cite: 95]. The `-gdb tcp::1234` flag opens a GDB server listening on port 1234[cite: 95]. When we type `make debug`, the terminal will show no program output; the emulator appears stalled because the CPU is powered on but intentionally waiting for a debugger to connect.

2. Connecting the Host (GDB): In a second terminal, we launch the debugger. It is vital to pass the ELF file as an argument when starting GDB (e.g., `arm-none-eabi-gdb build/scratch-os.axf`)[cite: 95]. GDB does not execute this file; it reads the metadata inside it[cite: 95]. Because we compiled our code with flags that preserve debugging symbols, GDB can map raw hexadecimal memory addresses back to human-readable C variables and function names (like `main`)[cite: 95]. Without the ELF file, the developer would only see raw addresses and would have to manually track the memory locations of every variable.

Note on Security: Keeping debugging symbols is essential during development. However, before releasing a final commercial product, these symbols are always stripped from the ELF file. Leaving them in provides malicious actors with a detailed blueprint of how the code operates, making it trivial to exploit.

Reconstructing the Debug Target

The `debug` target is essentially a shorthand, and you can reconstruct it for any image. Take the QEMU command from the `run` target of the `Makefile`, and append `-S -gdb tcp::1234`:

```
qemu-system-arm -machine mps2-an385 -monitor null -semihosting \
  --semihosting-config enable=on,target=native \
  -kernel build/scratch-os.axf -serial stdio -nographic \
  -S -gdb tcp::1234
```

This is important for future work: later checkpoints' `Makefiles` may have a `run` target but no `debug` target, and this command is how you will debug them.

The GDB Commands You Need

This small set of commands is enough for everything in this phase:

Command	What it does
<code>target remote :1234</code>	connect to the gdb server (QEMU) on this machine
<code>info registers sp pc</code>	show some registers (<code>info registers</code> alone: all of them)
<code>x/2wx 0</code>	examine memory: 2 words in hex, at address 0
<code>x/s &data_message</code>	examine memory as a C string
<code>p/x \$r1</code>	print one register (or a C expression) in hex
<code>display/i \$pc</code>	after every step, show the next instruction
<code>stepi (si)</code>	execute exactly one machine instruction (Enter repeats it)
<code>break main (b)</code>	stop when execution reaches <code>main</code> (or any label)
<code>continue (c)</code>	run until the next breakpoint; <code>Ctrl-C</code> interrupts it
<code>quit (q)</code>	leave gdb (QEMU keeps running: stop it with <code>Ctrl-C</code>)

Attaching at Reset

The first thing to do with the debugger is to catch the CPU *before* its first instruction, and check the reset contract. In the two terminals:

```
make debug                                # terminal 1: QEMU waits
arm-none-eabi-gdb build/scratch-os.axf    # terminal 2
```

And inside GDB:

```
(gdb) target remote :1234
(gdb) info registers sp pc
(gdb) x/2wx 0
```

You see `sp` equal to `0x20400000`, `pc` equal to `0x8 <Reset_Handler>`, and at address 0 the two words `0x20400000 0x00000009`. **No instruction has run yet**, yet `SP` and `PC` already have values: the CPU itself copied vector word 0 into `SP` and vector word 1 into `PC`.

Compare the two: word 1 is `0x9`, but `PC` is `0x8`. This is the **Thumb bit**. The CPU uses bit 0 to check that the target is Thumb code and then clears it; the instruction really starts at `0x8`. Why `0x8`? Because the base's vector table has only two words (8 bytes), and the code is placed right after it in FLASH.

2.3.4 Cortex-M3 Programmer's Model: The Registers

To follow the CPU instruction by instruction, you need to know what it holds. The **programmer's model** is the set of registers your code, and GDB, can see. All of them are 32 bits wide.

At this level, there are no variables: only these registers and memory. When your C code says `x = x + 1`, the compiler turns it into "load `x` from memory into a register, add 1 to the register, store the register back". The stack grows **downwards**: pushing a value first does `SP = SP - 4`, then writes the value at the new `SP`.

Register	Name	Role
R0–R12	general purpose	your values; a C function call passes its first arguments in R0–R3
R13	SP	stack pointer: address of the top of the stack
R14	LR	link register: where to return after a function call
R15	PC	program counter: address of the next instruction
—	xPSR	status: flags, Thumb bit, current exception

The Status Register (xPSR) and CPU Modes

xPSR (program status register) packs three critical pieces of information into one register:

- **Flags (Bits 31–28):** The **N Z C V** flags (negative, zero, carry, overflow), set by comparison instructions and used by conditional jumps.
- **Thumb Bit (Bit 24):** The **T** state bit, which must always be 1 on Cortex-M. This is where bit 0 of the vector entry ends up.
- **Exception Number (Bits 8–0):** Holds the number of the exception currently being handled, or 0 if none.

That last field dictates which of the two **modes** the CPU is in:

- **Thread mode:** The CPU runs normal code (`Reset_Handler`, `main()`, and the functions they call). The exception number is 0.
- **Handler mode:** The CPU runs an exception handler, and the field holds that exception's number.

Note: Code in Thread mode can also be privileged or unprivileged. All code in this specific unit runs privileged; the distinction will be explored in later sections.

2.3.5 ARM Assembly Basics: A Load/Store Architecture

The boot code must be written in assembly because, before it runs, there is no valid stack and no initialized data, meaning C code cannot yet be trusted. Fortunately, you need only a handful of instructions, and each has a direct C equivalent.

The ARM architecture is strictly a **load/store machine**[cite: 18]. Memory is touched only by **load** instructions (`ldr`, memory to register) and **store** instructions (`str`, register to memory)[cite: 18, 19]. Arithmetic and logic operations work only on registers[cite: 18].

The square brackets mean "the memory at this address". The form `[r0], #4` is called **post-increment**: the instruction first uses the address in `r0`, then adds 4 to `r0`[cite: 22]. Since one word is 4 bytes, a loop using this syntax walks through memory word by word, exactly like `*dst++` in C.

Assembly	C equivalent
<code>ldr r3, [r0]</code>	<code>r3 = *r0;</code>
<code>str r3, [r1]</code>	<code>*r1 = r3;</code>
<code>ldr r3, [r0], #4</code>	<code>r3 = *r0; r0 += 4; (i.e., r3 = *src++;)</code>
<code>str r3, [r1], #4</code>	<code>*r1 = r3; r1 += 4; (i.e., *dst++ = r3;)</code>
<code>strb r1, [r0, #7]</code>	<code>((uint8_t *)r0)[7] = r1; (one byte, at offset 7)</code>

Assembly	C equivalent
<code>ldr r0, =_estack</code>	<code>r0 = (uint32_t)&_estack; (any 32-bit value)[cite: 19]</code>
<code>mov sp, r0</code>	<code>sp = r0; (register to register)[cite: 19]</code>
<code>movs r3, #0</code>	<code>r3 = 0; (small constants only)[cite: 19]</code>
<code>copy_data:</code>	a label : a name for this address, like a C <code>goto</code> label
<code>b copy_data</code>	<code>goto copy_data;</code>
<code>bl main</code>	<code>main();</code> branch with link , saves return address in LR

Constants, Labels, and Jumps

Moving data between registers and controlling program flow requires another set of basic instructions:

The 32-bit Instruction Encoding Constraint: The `ldr rX, =value` form needs explanation. An instruction is only 16 or 32 bits long, so it mathematically cannot contain an arbitrary 32-bit constant alongside its opcode[cite: 19]. The assembler solves this by storing the constant in a small table near the code, called the **literal pool**[cite: 19]. It turns your `ldr r0, =_estack` into "load `r0` from that table". In GDB, this appears as something like `ldr r0, [pc, #92]` (a PC-relative address)[cite: 19].

The `.ltorg` directive at the end of the `Reset_Handler` tells the assembler where to put this literal pool: typically after the last instruction, where the CPU will never try to execute it as code.

2.3.6 Control Flow: Decisions, Bits, and Interrupts

Branching and loops in assembly rely entirely on the `xPSR` status register. A decision is a two-step process: compare values to set condition flags, then jump based on those flags[cite: 19].

Assembly	C equivalent
<code>cmp r1, r2</code>	compute <code>r1 - r2</code> , keep only the flags <code>N Z C V</code> [cite: 20]
<code>bhs done</code>	<code>if (r1 >= r2) goto done; (higher or same, unsigned)[cite: 20]</code>
<code>orr r1, r1, #0x10</code>	<code>r1 = 0x10;</code>
<code>cpsid i</code>	stop accepting interrupts (<code>__disable_irq()</code> in C)[cite: 21]
<code>cpsie i</code>	accept interrupts again (<code>__enable_irq()</code> in C)

With `cmp` and a conditional branch, you can build any loop. A C `while` loop becomes "test at the top, jump back at the bottom"[cite: 19, 20]:

```

loop:  cmp r1, r2          @ while (r1 < r2) {
        bhs done          @
        ...               @      body

```

```

        b    loop                @ }
done:

```

The loop continues while `r1 < r2`; as soon as `r1 >= r2`, `bhs` jumps out of the loop[cite: 20]. (*Note: `@` starts a comment in ARM assembly*).

2.3.7 Assembler Directives

Lines that start with a dot (.) produce no instructions: they steer the assembler itself[cite: 20]. The ones used in `boot/startup.s` are:

```

        .section .isr_vector,"a",%progbits    @ put what follows in section .i
        .global isr_vector                  @ make the name visible to the l
isr_vector:
        .word _estack                      @ one 32-bit value
        .word Reset_Handler

        .section .text,"ax",%progbits
        .global Reset_Handler
        .thumb_func                        @ next label is Thumb code (bit
Reset_Handler:

```

This vector table block means, in C equivalent:

```

__attribute__((section(".isr_vector")))
const uint32_t isr_vector[] = { (uint32_t)&_estack, (uint32_t)Reset_Hand

```

- `.section` chooses the section, with flags: "a" means the section occupies space in the image, "x" means it is executable.
- `.global` makes a name visible to other files and to the linker; without it, a label acts like a `static` name in C[cite: 20].
- `.word` allocates exactly one 32-bit word and stores the given value inside it[cite: 20].
- `.thumb_func` dictates that the next label is Thumb code[cite: 20]. This instructs the tools to store `Reset_Handler` with bit 0 set to 1, which the Cortex-M strictly requires[cite: 20].

Finally, the file also uses `.type` and `.size` directives to tell GDB and `nm` whether a name is code or data, and how big it is.

2.4 Writing the Startup Script (Reset_Handler)

The `Reset_Handler` in `boot/startup.s` is the very first piece of code executed by the CPU. Its one job is to turn a bare, uninitialized hardware chip into an environment fully capable of safely running C code[cite: 20].

Here is the complete plan, in the order the code does it:

In the base skeleton, `Reset_Handler` has only steps 6 and 7 (and without the unmask). You write everything else. In the rest of this unit, you go through the steps in order, together with the in-class checks that prove each one works.

Step	Why
1. mask interrupts	nothing may interrupt a half-built system
2. set SP	C functions need a stack
3. copy <code>.data</code> FLASH $\rightarrow$ RAM	initialized globals get their values
4. zero <code>.bss</code>	C promises uninitialized globals start at 0
5. configure the CPU	vector table address, fault handlers, traps, priorities
6. unmask interrupts, <code>bl main</code>	hand over to C
7. loop forever	if <code>main</code> ever returns, there is nowhere to go

2.4.1 Steps 1 and 2: Mask Interrupts and Set the Stack

An **interrupt** is an exception raised by hardware (e.g., a timer expiring or a device receiving data): the CPU drops what it is doing and runs that interrupt's handler[cite: 21]. During setup, nothing is ready to handle one: a handler written in C could read a variable that has not been initialized yet.

So the very first instruction, `cpsid i`, tells the CPU to ignore interrupts for now. (No device has been told to raise an interrupt yet, so this is not strictly needed at this moment; writing it makes the rule explicit instead of relying on luck.)

Then, the stack pointer (SP) must be set:

```
C:      sp = &_estack;          ASM:  ldr r0, =_estack    @ top of RAM, 0x20000000
                                   mov sp, r0
```

C has no way to name SP directly, so the C column is pseudo-code. You might wonder why this is needed, since the hardware already loaded SP from word 0 of the vector table at reset[cite: 21]. The answer is that `Reset_Handler` is not always reached through a real reset: a bootloader or a debugger may jump to it directly, with SP holding any random value[cite: 21].

Setting SP explicitly makes the code correct in every case, and shows that the stack is nothing more than an address in a register. Notice the two-step pattern: SP is a special register, so the address is first loaded into an ordinary register (`r0`), and then copied into SP with `mov`[cite: 21].

2.4.2 Step 3: Copy `.data` from FLASH to RAM

This is the step that fixes the base skeleton's missing print line[cite: 21]. The linker script gives you three addresses:

```
FLASH:  [ .text | .rodata | initial values of .data |
          ^ _sidata
RAM:     [ .data          | .bss | ...          stack |
          ^ _sdata        ^ _edata          ^ _estack
```

The copy is a simple loop, which you can write first in C and then translate line by line into assembly[cite: 22]:

```
C                               ASM
uint32_t *src = &_sidata;      ldr r0, =_sidata
uint32_t *dst = &_sdata;        ldr r1, =_sdata
                                ldr r2, =_edata
while (dst < &_edata)          copy_data: cmp r1, r2
```

```

                                bhs copy_data_done
*dst++ = *src++;                ldr r3, [r0], #4
                                str r3, [r1], #4
                                b    copy_data
                                copy_data_done:

```

Register `r0` plays the role of `src`, `r1` of `dst`, and `r2` holds the end address. The loop is `copy_data`, and it exits to `copy_data_done`. It copies whole words, 4 bytes at a time; this is safe because the linker script aligns the start and end of `.data` to multiples of 4.

Try It: Bring Data to Life

In class, you write steps 1-3 into the base skeleton, just before `bl main`, and run it again:

```
make run
```

And the missing line finally appears:

```
main: hello from the base
main: this text lives in .data
```

Before the copy, `data_message` sat in RAM as all zeros and printed as an empty string; now it successfully holds its initial value[cite: 22].

Watching the Copy Loop in GDB

It is worth watching your loop run because it makes the idea of LMA and VMA concrete. Start `make debug` in terminal 1 and GDB in terminal 2, then:

```

(gdb) target remote :1234
(gdb) display/i $pc
(gdb) stepi                               # press Enter to repeat until "cmp r1, r2"
(gdb) p/x $r0                             # src: a FLASH address
(gdb) p/x $r1                             # dst: 0x20000000, start of RAM
(gdb) x/s &data_message                   # "" : nothing copied yet
(gdb) stepi 5                             # one full turn of the loop
(gdb) p/x $r1                             # 0x20000004
(gdb) break copy_data_done
(gdb) continue
(gdb) x/s &data_message                   # "main: this text lives in .data\n"

```

As you step, `display/i $pc` shows each instruction just before it runs. When you reach `cmp`, `r1` holds `0x20000000`, and the string is empty. Five `stepi` commands execute one full turn of the loop, moving `r1` to `0x20000004`. By continuing to the breakpoint at the end, `r1` climbs until it equals `r2` (`0x20000020`), and only then does the string appear in RAM.

Try It: Breaking the Stack on Purpose

Seeing a wrong SP go wrong is the best way to believe that SP matters. In `boot/startup.s`, replace `_estack` with a bad address:

Test 1: Address Outside of RAM (0x30000000) If you set SP to 0x30000000, nothing prints[cite: 23]. QEMU stops with:

```
qemu: fatal: Lockup: can't escalate 3 to HardFault (current priority -1)
```

When `main` starts, it pushes registers to the stack (below 0x30000000), triggering a memory fault[cite: 23]. The CPU tries to run a fault handler to report it, but entering a handler requires writing to the stack, which fails again[cite: 23]. Unable to report the fault, the CPU gives up and enters a **Lockup** state[cite: 23].

Test 2: Unaligned Stack Pointer (0x20000001) If you force SP to an odd address inside RAM, `make run` prints nothing[cite: 23]. If you inspect this in GDB (`info registers sp`), it says 0x20000000. The CPU silently drops the two lowest bits of SP because SP must always be a multiple of 4[cite: 23]. The stack now starts at the bottom of RAM and grows *below* it into invalid memory[cite: 23]. The function calls fail silently.

2.4.3 Step 4: Zero the .bss Section

The C standard promises that global variables without an initializer (`int counter;`) start at 0[cite: 23]. They live in `.bss`, between `_sbss` and `_ebss` in RAM, and nothing in flash holds their value: the boot code must actively write the zeros[cite: 23].

It is the same loop as the copy, but without a source pointer:

C	ASM
<code>uint32_t *dst = &_sbss;</code>	<code>ldr r1, =_sbss</code>
	<code>ldr r2, =_ebss</code>
	<code>movs r3, #0</code>
<code>while (dst < &_ebss)</code>	<code>zero_bss: cmp r1, r2</code>
	<code>bhs zero_bss_done</code>
<code> *dst++ = 0;</code>	<code>str r3, [r1], #4</code>
	<code>b zero_bss</code>
	<code>zero_bss_done:</code>

This is `zero_bss` in `boot/startup.s`. In the current program, `.bss` happens to be empty, so the loop exits immediately, but the first global you add will rely on it. Remember that RAM is **not** zero at power-on on real hardware; QEMU zeroing it for you is a convenience you must not count on!

2.4.4 Exceptions and Faults: One Family

With steps 1-4, `Reset_Handler` does everything that the previous C startup code did. The remaining steps go further: they configure the CPU so that errors are caught and reported.

You have already met the word **exception**: any hardware event that makes the CPU stop the current code and run a specific handler function in Handler mode[cite: 24]. The Cortex-M3 defines the following family of exceptions:

Apart from the first two slots, all remaining slots in the vector table are associated with these exceptions[cite: 24]. Specific fault handlers (like `MemManage`, `BusFault`, and `UsageFault`) are crucial for both debugging and security[cite: 24].

#	Exception	Raised by
0	<i>(not an exception)</i>	this slot holds the initial SP value
1	Reset	power-on, reset button
2	NMI	non-maskable interrupt: cannot be ignored[cite: 24]
3	HardFault	any fault with nowhere else to go[cite: 24]
4	MemManage	breaking memory protection rules[cite: 24]
5	BusFault	access to an address where nothing answers, or refuses access[cite: 24]
6	UsageFault	bad use of an instruction: undefined opcode, Thumb bit 0, divide by 0[cite: 24]
7–10	<i>reserved</i>	unused: the slot must hold 0
11	SVCall	the <code>svc</code> instruction
12	DebugMonitor	debug events (unused here)
13	<i>reserved</i>	unused: the slot must hold 0
14	PendSV	a request raised by software
15	SysTick	the CPU's built-in timer
16+	IRQs	device interrupts

Completing the Table with Weak Symbols

The base table has 2 words, but the CPU expects one word for each exception from 0 to 15 (16 words in total). Every slot needs *some* valid address, even for exceptions you have no handler for yet. The reserved slots (7–10 and 13) must contain 0.

To achieve this without writing empty functions for every exception, a best practice is to use a generic template leveraging **weak symbols** and aliases in assembly:

```
.word HardFault_Handler @ slot 3, and so on up to
.weak HardFault_Handler @ may be replaced from C
.thumb_set HardFault_Handler, Default_Handler
Default_Handler:
    b Default_Handler @ for (;;) {}
```

Which in C (with a GCC extension) translates to:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")))
```

How it works:

1. `.word HardFault_Handler` places the address of the label into the table slot.
2. `.thumb_set` tells the assembler: "Make `HardFault_Handler` another name for `Default_Handler` (same address, marked as Thumb code)."
3. `.weak` makes that name a **weak symbol**: a default that gives way. If a developer writes a normal, non-weak C function named `void HardFault_Handler(void)`, the linker silently drops the assembly default and routes the exception directly to the developer's C function.

`Default_Handler` itself is an infinite loop: an unexpected exception is a bug, and freezing at a known place is much easier to diagnose with a debugger than running off into random memory.

The Effect on Memory Layout Completing the table moves your code. The table grows from 2 words (8 bytes) to 16 words (64 bytes). Because the executable code still comes right after it in FLASH, `Reset_Handler` moves from address `0x8` to `0x40`. If you attach GDB at reset now, `info registers pc` shows `0x40`, and examining address `0` (`x/2wx 0`) prints `0x20400000 0x00000041` (the Thumb bit added to `0x40`).

2.4.5 Step 5: Configure the CPU (System Control Block)

The remaining configuration steps change settings inside the CPU core itself. You do not need special instructions for most of them: the Cortex-M makes these settings available as 32-bit registers at fixed memory addresses, in the system area of the memory map (`0xE000E000–0xE000EFFF`).

The group of registers that controls exceptions is called the **System Control Block (SCB)**. You change them with ordinary loads and stores:

C	ASM
<code>#define SCB_SHCSR (*(volatile uint32_t *)0xE000ED24)</code>	
<code>SCB_SHCSR = (1u << 16);</code>	<code>ldr r0, =0xE000ED24 @ address</code>
	<code>ldr r1, [r0] @ read</code>
	<code>orr r1, r1, #0x10000 @ modify</code>
	<code>str r1, [r0] @ write</code>

Read-Modify-Write and the `volatile` Keyword

This pattern is called **read-modify-write**: you read the current value, set only the bits you care about, and write the result back, so that all the other bits keep their original values.

The `volatile` keyword in the C version is absolutely critical. It tells the compiler that every memory access must really happen exactly in the order written. The compiler may not skip a read because "the value can't have changed", nor remove a write because "nobody reads it". For hardware registers, both of those optimization assumptions are false.

You can read these registers from GDB like any memory. For example, `x/wx 0xE000ED00` shows `0x410fc231`, the CPUID register, which identifies the CPU as an ARM Cortex-M3.

VTOR: Vector Table Offset Register

When an exception happens, the CPU reads the vector table. The **VTOR** register (at `0xE000ED08`) holds the memory address of the table. Its value at reset is `0x00000000`, which is why our initial table must be at address 0.

However, in real systems, the table is often moved (e.g., a bootloader starting an application higher in flash, or a program copying its table into RAM to modify handlers). Writing VTOR explicitly makes the boot code robust wherever the image is placed:

C:	<code>*(volatile uint32_t *)0xE000ED08 = (uint32_t)isr_vector;</code>
ASM:	<code>ldr r0, =0xE000ED08</code>

```
ldr r1, =isr_vector
str r1, [r0]
```

Because we are setting the entire 32-bit register, no read-modify-write is needed. (*Note: The table address must be aligned to at least 128 bytes*).

SHCSR: System Handler Control and State Register

By default at reset, specific faults (MemManage, BusFault, UsageFault) are disabled, meaning all faults escalate to HardFault. You get a single handler for every kind of error, making it impossible to tell *what* went wrong.

The **SHCSR** register (at 0xE000ED24) enables these specific faults:

Bit	Name	When set
16	MEMFAULTENA	MemManage faults run MemManage_Handler
17	BUSFAULTENA	bus faults run BusFault_Handler
18	USGFAULTENA	usage faults run UsageFault_Handler

The boot code must set bits 16–18 (the value 0x00070000) using the read-modify-write pattern, because the other bits of SHCSR report status (which handlers are active or pending) and must not be overwritten.